FlashSAC, Explained

A complete plain-English guide to the paper,
and exactly how it differs from SAC v2

Covering: Kim, Lee, Park, Kim, Nahendra, Seno, Min, Palenicek, Vogt, Kragic, Peters, Choo & Lee (2026).
FlashSAC: Fast and Stable Off-Policy Reinforcement Learning for High-Dimensional Robot Control. arXiv:2604.04539v2 [cs.LG].

Study guide prepared for an RL reading group.

Contents

Part 0 How to use this document

This guide explains the FlashSAC paper completely, in ordinary language, and then places it side by side with SAC v2 so the differences are unambiguous.

What "SAC v2" means here

Soft Actor-Critic exists in two published versions, and people are often loose about which one they mean.

- **SAC v1** is Haarnoja et al., *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*, ICML 2018 (arXiv:1801.01290). It had a separate state-value network V and a fixed entropy temperature that you tuned by hand for every task.
- **SAC v2** is Haarnoja et al., *Soft Actor-Critic Algorithms and Applications* (arXiv:1812.05905). It deleted the V network, kept two Q networks plus two slowly-updated target Q networks, and made the entropy temperature learn itself. This is the version that every modern library implements when it says "SAC".

When this document says SAC v2 it means the second one. Note that FlashSAC's own background section describes precisely SAC v2 (two Q functions, exponential-moving-average target networks, a learned temperature), even though its citation points at the ICML paper. So SAC v2 really is the correct baseline for the comparison, not a strawman.

Where the content comes from

Everything stated as a fact about the method, the experiments, or the numbers comes from the paper itself, including its appendices. Three kinds of content are marked separately:

- Boxes labelled **Derived** contain arithmetic done here from the paper's numbers. The result is not printed in the paper, but it follows directly from its stated values, and it is usually the most useful thing to say out loud in a reading group.
- Boxes labelled **Watch out** flag a caveat, an ambiguity, or an internal inconsistency in the paper.
- The SAC v2 reference configuration in Part 3 comes from the SAC v2 paper and from what standard implementations do. Individual codebases vary slightly.

Part 1 The whole paper in one page

The claim. For roughly five years, the default way to train a robot in simulation and deploy it on hardware has been PPO, an on-policy algorithm. FlashSAC argues that this default was an accident of working on *small* robots. Once you properly control how the critic updates itself, off-policy Soft Actor-Critic beats PPO on both axes at once, final performance and wall-clock training time, as soon as the robot gets big. They demonstrate this on more than 60 tasks across 10 simulators, and on a real 29-joint humanoid.

Question	Answer
What is the base algorithm?	SAC v2. The learning rule is unchanged. FlashSAC changes the network, the data regime, the critic's output format, and the exploration noise.
What is the core problem?	Off-policy RL sees more diverse data than on-policy RL, which should help. But extracting value from that data normally needs many gradient updates, which is slow and which compounds the critic's own errors through bootstrapping.
What is the core idea?	Take the scaling recipe from supervised learning: a bigger model with bigger batches and *far fewer* update steps. That recipe normally destroys off-policy RL, because bigger critics are less stable under bootstrapping. So first make the critic structurally incapable of blowing up, by bounding its weight norms, feature norms and gradient norms. Then the recipe works.
What are the three pillars?	(1) Fast: 1024 parallel simulators, a 10M-transition buffer, a 2.5M-parameter network, batch 2048, and only 2 gradient steps per 1024 collected transitions. (2) Stable: inverted residual blocks, pre-activation BatchNorm with a shared-statistics trick, RMSNorm before the value head, a distributional critic with automatic reward scaling, and weight projection onto the unit sphere. (3) Exploration: an entropy target expressed as an action standard deviation, plus repeated action noise.
What is the headline result?	On a real Unitree G1 humanoid, stable flat-ground walking after about 20 minutes of training, against about 3 hours for PPO. Real stair climbing after about 4 hours, against nearly 20 hours for PPO.
What is the honest limitation?	On low-dimensional tasks FlashSAC is only comparable to PPO, not better. The wall-clock plots are partly reconstructed rather than measured end to end. The sim-to-real comparison uses different reward weights for the two algorithms. And the paper is a synthesis of existing techniques rather than a new theoretical result.

THE ONE-SENTENCE DIFFERENCE FROM SAC V2

FlashSAC keeps SAC v2's mathematics exactly and replaces everything around it: a much larger normalized network instead of a small plain MLP, a thousand simulators instead of one, a classification-style distributional critic with normalized rewards instead of scalar regression, roughly 64 times *fewer* gradient touches per collected sample, and temporally repeated exploration noise instead of fresh noise every step.

Part 2 The background you need

2.1 The vocabulary, defined once

Reinforcement learning models the robot's world as a **Markov Decision Process**, written $M = (S, A, P, r, \gamma)$. S is the set of states, A is the set of actions (here continuous, for example 29 joint targets), $P(s' \mid s, a)$ is the physics, $r(s, a)$ is the reward, and γ between 0 and 1 says how much you care about the future relative to now.

At each timestep the agent sees a state, picks an action, gets a reward, and lands in a new state. The goal is a **policy** $\pi(a \mid s)$, a rule for choosing actions, that maximizes the discounted sum of future rewards.

Modern continuous-control algorithms use two networks:

- The **actor** is the policy itself. It maps a state to an action.
- The **critic** $Q(s, a)$ estimates how much total future reward you will get if you take action a in state s and behave well afterwards. The actor improves by climbing the critic's estimate.

Two more terms that appear constantly in this paper:

- A **replay buffer** is a large memory of past transitions (s, a, r, s') that the algorithm can sample from repeatedly.
- The **update-to-data ratio** (UTD, sometimes called the replay ratio) is how many gradient updates you perform per new transition collected from the environment. It is the single most important dial in this paper.

2.2 On-policy versus off-policy

This is the fault line the entire paper sits on.

	On-policy (PPO)	Off-policy (SAC, TD3)
How data is used	Collect a batch with the current policy, take a few gradient steps on it, then delete it forever.	Every transition ever collected goes into a replay buffer and can be sampled again at any time.
Why it is stable	The data always matches the policy being evaluated. There is no mismatch to correct for.	It is not automatically stable. The critic must judge actions the current policy would never take.
What it costs	Every single gradient step needs fresh simulation.	Nothing extra in simulation, but a great deal of care in how the critic is trained.
Where it wins	Simulation is cheap and the robot is small.	Simulation is expensive, or the robot is large, or both.

An analogy. On-policy learning is a student who studies only from today's lecture notes and burns them every evening. Nothing they study is out of date, but they can only ever learn from what happened today. Off-policy learning is a student who keeps every notebook they have ever filled. They can revisit anything, but they must be careful not to trust an old note that has since been proven wrong.

2.3 Bootstrapping, and why it is dangerous

The critic is trained by regression, but there is no ground-truth label for "how good is this state". So the algorithm builds a label out of its own prediction. This is **bootstrapping**. The target for the pair (s, a) is

$$y = r + \gamma \cdot \bar{Q}(s', a'), \quad \text{where } a' \text{ is what the current policy would do in } s'$$

and the critic is trained to make Q(s, a) match y. Notice that $\bar{Q}$ is the critic itself, in a slowly updated copy. The critic is grading its own homework against an answer key it also wrote.

The consequence is that errors do not simply sit there. If the critic overestimates the value of some state-action pair (s′, a′), that overestimate is injected into the target for (s, a), which trains the critic to overestimate (s, a) too, which then poisons whatever bootstraps off (s, a), and so on. Errors are recycled and amplified, not averaged away.

The combination of three ingredients is known in the literature as the **deadly triad**: function approximation (a neural network), bootstrapping (targets built from your own predictions), and off-policy data (training on actions your current policy would not take). Off-policy deep RL has all three at once. This is the single fact you need in order to understand why half of FlashSAC is normalization layers.

WHY THIS GETS WORSE IN HIGH DIMENSIONS

The action a′ in the target is sampled from the current policy. In a 6-joint arm, most sampled actions are similar to actions already in the buffer, so the critic is interpolating. In a 29-joint humanoid or a 20-joint robot hand, the policy will constantly propose action vectors that are far from anything the critic has seen. The critic then *extrapolates*, and neural networks extrapolate badly and confidently. Those confident wrong values go straight into the target. This is why the paper says instability grows with both state-action dimensionality and model capacity.

2.4 The three chronic problems of off-policy RL

The paper's related-work section is organised around three failure modes, and it is worth holding on to them because FlashSAC's three pillars map onto them one for one.

1. **It is slow.** Fitting an accurate critic over a diverse buffer normally needs a great many gradient updates. Each update costs wall-clock time.
2. **It is unstable.** Each of those many updates is another round of the error recycling described above.
3. **Exploration is hard in high-dimensional action spaces.** SAC's entropy bonus alone is not enough to produce coherent exploratory *behaviour* when the action vector is long.

The paper's observation is that prior work attacked these one at a time. Speed-focused work (FastTD3, FastSAC) used tiny networks to stay stable, which caps final performance. Stability-focused work (CrossQ, Simba, SimbaV2, BRO, XQC) enabled bigger networks, but bigger networks needed *more* updates, which put the speed back. Exploration work left both bottlenecks untouched. FlashSAC's contribution is doing all three simultaneously, which is what makes the combination more than the sum of its parts.

2.5 Why robotics defaulted to PPO, and why that is now changing

Sim-to-real RL has been most successful on quadruped locomotion and simple gripper manipulation. Those domains share two properties: low-dimensional state and action spaces, and extremely fast simulators. In that setting PPO's data inefficiency simply does not matter, because fresh data is nearly free, and its stability and ease of tuning are decisive advantages.

The paper argues that this regime is no longer representative. Three things changed at once:

- Humanoid locomotion, dexterous in-hand manipulation and vision-based control all have far larger state and action spaces, so a single policy's rollouts cover a vanishingly small slice of what matters.
- Simulation is getting more expensive, because contact-rich and deformable physics is expensive.
- Policies are getting bigger, so even the rollout (inference) cost is rising.

When fresh data stops being cheap and one policy's data stops being representative, throwing that data away after a handful of gradient steps becomes an expensive habit. That is the opening FlashSAC walks through.

Part 3 SAC v2, precisely

To say clearly what FlashSAC changed, we first need SAC v2 pinned down exactly. This part is the baseline. If you already know SAC cold, skim the equations and go straight to the configuration table in 3.4, because that table is what Part 5 compares against.

3.1 The two SAC papers, and what v2 changed

Aspect	SAC v1 (arXiv:1801.01290)	SAC v2 (arXiv:1812.05905)
Networks	One policy, two Q functions, one state-value function V, and a target copy of V.	One policy, two Q functions, and two slowly updated target Q functions. V is deleted entirely.
Entropy temperature α	A fixed number you tune per task, usually indirectly by scaling the reward.	Learned automatically by gradient descent so that the policy's entropy tracks a chosen target.
Practical effect	Extremely sensitive to reward scale. Retuning per task was mandatory.	Roughly one configuration works across many tasks. This is why v2 became the default.

3.2 The maximum-entropy objective

Ordinary RL maximizes expected return. SAC maximizes return *plus* the randomness of the policy:

$$\text{maximize} \quad E\left[\Sigma_t \gamma^t \left(r(s_t, a_t) + \alpha \cdot H(\pi(\cdot \mid s_t)) \right) \right]$$

H is **entropy**, a measure of how spread out the policy's action distribution is. A policy that always outputs the same action has low entropy. A policy that samples widely has high entropy. The temperature α sets the exchange rate between reward and randomness.

Two reasons this matters, and the second one is the one people forget:

- It keeps the agent exploring instead of locking onto the first mediocre behaviour it finds.
- It keeps the replay buffer diverse. A buffer filled by a nearly deterministic policy gives the critic very narrow data, which makes the extrapolation problem from Part 2.3 worse. Entropy is therefore not just an exploration device in SAC, it is a data-quality device.

3.3 The three loss functions, term by term

The critic loss

SAC keeps two critics, $Q_{\varphi 1}$ and $Q_{\varphi 2}$, and each is trained to match a bootstrapped target:

$$L(\varphi_i) = E_{(s,a,r,s') \sim D}\left[Q_{\varphi i}(s, a) - y \right]^2$$

$$y = r + \gamma \left(\min_{j=1,2} Q_{\bar{\varphi} j}(s', a') - \alpha \log \pi_\theta(a' \mid s') \right), \quad a' \sim \pi_\theta(\cdot \mid s')$$

Three things are happening in that target.

- **min over two critics.** This is **clipped double Q-learning**, taken from TD3. Because the target involves a maximization over actions, any random overestimation in the critic gets selected for and grows. Taking the smaller of two independently initialized critics deliberately biases the estimate downward, which is far safer than biasing it upward.
- **minus α log π.** This is the entropy bonus written in sampled form. An action the policy considers unlikely has a very negative log-probability, so subtracting it adds value. This is what makes the backup a "soft" Bellman backup.
- **$\bar{\varphi}$, the target networks.** Slowly moving copies of the critics, updated after every step by an exponential moving average, $\bar{\varphi} \leftarrow \tau\varphi + (1 - \tau)\bar{\varphi}$. With $\tau = 0.005$ the target lags roughly 200 updates behind. Their whole purpose is to stop the critic chasing a target that moves as fast as it does.

The actor loss

$$L(\theta) = E_{s \sim D,\, a \sim \pi\theta} \left[\alpha \log \pi_\theta(a \mid s) - \min_{i=1,2} Q_{\varphi i}(s, a) \right]$$

In words: pick actions the critic rates highly, but stay random. The gradient flows through the sampled action into the critic using the reparameterization trick, so the actor is literally doing gradient ascent on the critic's surface. The policy is a Gaussian passed through a tanh, which keeps actions inside the valid range.

The temperature loss (the v2 addition)

$$L(\alpha) = E_{a \sim \pi} \left[-\alpha \left(\log \pi(a \mid s) + \bar{H} \right) \right]$$

Read this as a thermostat. $\bar{H}$ is the target entropy you want the policy to have. If the policy is currently more deterministic than the target, α rises, which increases the pressure to be random. If it is more random than the target, α falls. You no longer tune α, you tune $\bar{H}$. SAC v2 proposes the heuristic $\bar{H} = -|A|$, one negative nat per action dimension. Part 4.3 shows exactly what that number means physically, and why FlashSAC replaces it.

3.4 The standard SAC v2 configuration

This is the reference point for every comparison later in the document.

Setting	Value	Note
Parallel environments	1	Occasionally a handful; the algorithm assumes data is scarce.
Replay buffer size	1,000,000	The near-universal default.
Batch size	256	
Update-to-data ratio	1	One gradient update per environment step.
Network	MLP, 2 hidden layers of 256, ReLU	Roughly 0.1 to 0.2M parameters. Same size for actor and critic.
Normalization layers	None	No BatchNorm, no LayerNorm, no RMSNorm.
Residual connections	None	Plain feedforward stack.
Weight constraints	None	Weight norms grow freely during training.

Setting	Value	Note
Critic output	One scalar per critic	Trained with mean squared error.
Reward handling	Raw rewards	No normalization. Scale sensitivity is absorbed by learned α.
Number of critics	2, minimum taken	Clipped double Q.
Target smoothing τ	0.005	Updated every step.
Actor update frequency	Every critic update	No delay.
Target entropy $\bar{H}$	$-\|A\|$	Fixed heuristic.
Initial α	1.0	Typically; logα initialized at 0.
Optimizer / learning rate	Adam, 3×10^{-4}, constant	No schedule.
n-step returns	n = 1	Single-step Bellman backup.
Exploration noise	Fresh sample from the policy every step	Independent across timesteps.
Discount γ	0.99	

3.5 Where SAC v2 breaks down on big robots

Nothing in the table above is wrong. It is a well-designed algorithm for the regime it was built for: one simulator instance, scarce data, moderate action dimension. Four specific things stop working when you move to a 29-joint humanoid with a thousand parallel simulators.

1. **The network is too small to absorb the data.** A 2×256 MLP fitting a value function over ten million diverse transitions in a 300-dimensional observation space is underpowered. But making it bigger without other changes makes it diverge, which is precisely why the field kept the networks small.
2. **UTD = 1 is impossibly slow at scale.** With 1024 parallel environments you collect 1024 transitions per environment step. UTD = 1 would mean 1024 gradient updates per environment step. That is not a training run, that is a heat source.
3. **Nothing bounds anything.** No normalization means feature magnitudes, weight norms and Q-value magnitudes can all grow without limit. In the deadly-triad setting this is the mechanism by which training silently diverges.
4. **The target entropy does not transfer.** $\bar{H} = -|A|$ is a number in nats. It says nothing about how much the joints will actually move, and its physical meaning changes with the action space, so it needs retuning per embodiment.

Part 4 FlashSAC, component by component

4.0 The logical chain

Before the details, hold on to the argument, because the three pillars are not independent improvements. They are links in one chain, and every link is load-bearing.

> **THE ARGUMENT IN FIVE STEPS**
>
> 1. Bound the critic's update dynamics so it structurally cannot blow up.
> 2. Because it cannot blow up, you can afford a much bigger critic.
> 3. A bigger model with a bigger batch converges in far fewer update steps. This is the standard scaling behaviour observed in supervised learning.
> 4. Fewer update steps means less wall-clock time *and* fewer rounds of bootstrap error recycling. Speed and stability stop being a trade-off and start reinforcing each other.
> 5. Add off-policy data coverage on top, and you beat PPO on high-dimensional tasks.

If you remove step 1, the big model diverges. If you remove step 2, you are back to a small network that needs many updates. This is why the paper insists that stability is a *prerequisite* for scaling rather than a nice-to-have, and that framing is the paper's real intellectual contribution.

4.1 Pillar 1: fast training

Massively parallel simulation

Data is collected from 1024 simulated robots running at once on the GPU. Standard off-policy setups use one. The point is not just raw throughput. With 1024 environments carrying different random seeds, different terrain and different domain randomization, each collection round deposits a genuinely diverse slice of the state-action space into the buffer, rather than 1024 more steps of one trajectory.

Large-capacity replay buffer

The buffer holds up to 10 million transitions, ten times the usual one million. The stated reason is long-tail preservation. In a high-dimensional task, the rare but critical state-action pairs (the recovery step after a stumble, the awkward finger regrasp) are exactly the ones a small circular buffer overwrites first. Losing them causes two problems at once: the policy forgets the behaviour, and the critic loses the only data that anchored its value estimate in that region, which turns a previously interpolated value into an extrapolated one.

Large model, large batch, far fewer updates

This is the heart of pillar 1.

Knob	FlashSAC	Conventional off-policy
Parameters	2.5M total, 6 layers	0.2 to 0.5M, 2 to 3 layers
Batch size	2048	256
Update-to-data ratio	2 updates per 1024 new transitions	1 update per transition

Knob	FlashSAC	Conventional off-policy
Learning rate	3×10^{-4}, cosine decay to 1.5×10^{-4}	3×10^{-4}, constant

The batch of 2048 is chosen to nearly saturate the GPU, meaning the arithmetic units are busy rather than waiting on memory. A batch that large is close to free per sample. The reasoning is the same as in supervised learning: under a fixed compute budget, a large model taking a few well-informed steps beats a small model taking many noisy ones.

DERIVED: HOW OFTEN IS EACH TRANSITION ACTUALLY USED?

Per 1024 newly collected transitions, FlashSAC does 2 gradient steps with batch 2048. That is 2 × 2048 = 4096 samples drawn per 1024 samples collected, so each transition is used on average **4 times** over its entire life in the buffer.

You can verify this the other way round. A transition survives 10,000,000 / 1024 ≈ 9766 collection rounds, during which 19,532 updates happen, each drawing 2048 out of 10 million. Expected draws = 19,532 × 2048 / 10,000,000 ≈ 4.0. The two calculations agree.

For comparison, standard SAC v2 (one environment, UTD 1, batch 256) draws 256 samples per collected transition. PPO takes roughly 5 epochs over its rollout and then deletes it, so about 5 touches.

The implication. FlashSAC touches each sample about as often as PPO does, and roughly 64 times less than standard SAC. So the advantage is *not* squeezing each sample harder. The advantage is that the sample *stays in the buffer*, so every batch is drawn from a mixture of thousands of past policies instead of from one. Off-policy learning here buys distributional diversity, not sample reuse. Most people's mental model of "off-policy equals data efficient equals replay each sample many times" is exactly backwards for this paper, and saying so is usually the moment a reading group sits up.

WATCH OUT: AN INCONSISTENCY IN THE PAPER

Section 4.1 states the update-to-data ratio as 2/1024 and explains it as "2 gradient updates per 1024 new transitions". Table 9 in the appendix lists it as 2/2048. These cannot both be the denominator of the same quantity. The section-4.1 wording is self-consistent (1024 environments, one step each, equals 1024 new transitions), so that is the reading used throughout this document. If someone in the group spots it, the honest answer is that it is a documentation slip and it does not change any conclusion, but it does change the derived replay ratio by a factor of two.

Code optimization

Implemented in PyTorch, with both training and inference just-in-time compiled to cut Python interpreter overhead, and mixed-precision arithmetic throughout, which the paper credits with 5 to 10 percent of wall-clock time. Worth mentioning because it is a reminder that a claimed speedup in this literature is part algorithm and part engineering.

4.2 Pillar 2: stable training

Every component here bounds one specific quantity. Presenting them as a list of tricks is what makes this section feel arbitrary. Presenting them as four quantities under control is what makes it feel designed.

Quantity bounded	Mechanism	What goes wrong without it
Gradient norm	Inverted residual blocks	Gradients vanish or explode as depth grows, so you cannot stack 6 layers.
Feature norm	RMSNorm before the value head	An out-of-distribution (s′, a′) produces an enormous activation, hence an enormous Q, which poisons the target.
Activation scale and conditioning	Pre-activation BatchNorm, plus cross-batch value prediction	Activations saturate under shifting input distributions; and BatchNorm alone silently breaks the Bellman equation.
Target noise and scale	Distributional critic with adaptive reward scaling	Squared error on a noisy scalar target produces large, outlier-driven gradients, and one reward scale per task means one hyperparameter set per task.
Weight norm	Projection onto the unit sphere after every step	Q-value sensitivity grows, and the effective learning rate silently decays to zero.

Inverted residual backbone

The network stacks blocks borrowed from the Transformer feed-forward layer. Each block takes a feature vector of width d, expands it to 4d, applies a nonlinearity, projects back down to d, and adds the block's input back on as a **residual connection**.

> x → BatchNorm → Linear(d → 4d) → BatchNorm → ReLU → Linear(4d → d) → BatchNorm → ReLU → + x

The expand-then-contract shape is the **inverted bottleneck** from MobileNet. It gives the block a lot of representational capacity in the middle while keeping the interface between blocks narrow, so parameters are spent where they do work.

The residual connection is the part that matters for stability. It gives the gradient a direct path from the loss back to every earlier layer, so gradient magnitude does not decay or blow up multiplicatively with depth. This is what makes 6 layers possible when the field's default was 2 or 3.

DERIVED: WHERE THE 2.5M PARAMETERS AND THE 6 LAYERS COME FROM

Critic width 256 with 2 blocks: each block is Linear(256→1024) plus Linear(1024→256), which is 262,144 + 262,144 = 524,288 parameters, so 2 blocks is about 1.05M. Add the input projection and the 101-way output head and you get roughly 1.1M per critic, so about 2.2M for the pair. The actor at width 128 with 2 blocks is about 0.26M. Total roughly 2.5M, matching the paper. The "6 layers" is the linear-layer count along one path: 1 input projection + 4 linears inside the two blocks + 1 output head.

Post-block RMSNorm

After the final residual block, and immediately before the value head, the paper applies **RMSNorm**: divide the feature vector by its own root-mean-square magnitude. The effect is that whatever comes out of the backbone lands on a sphere of fixed radius.

This is the most direct defence against extrapolation error. Recall that the bootstrapped target requires evaluating $\bar{Q}(s', a')$ at action vectors the policy has just invented, some of which are genuinely outside anything in the buffer. Without a bound, a strange input can produce a huge

activation, hence a huge Q-value, which becomes a huge target, which the critic then trains towards. RMSNorm makes that failure structurally impossible: the head's input magnitude is fixed, so its output range is bounded no matter how strange the input.

Pre-activation BatchNorm, and why not LayerNorm

BatchNorm is applied before each nonlinearity. The problem it solves is that replay data is collected by a constantly changing mixture of policies, so the distribution of inputs to the network keeps shifting. Without normalization, activations drift into saturated regions and units go permanently inactive, the phenomenon usually discussed under **plasticity loss** or **dormant neurons**: the network stops being able to learn new things even though gradients are still flowing.

The interesting choice is BatchNorm rather than LayerNorm. Most recent scaled-RL work (Simba, BRO) uses LayerNorm, precisely because BatchNorm has a reputation for breaking RL. The paper's reasoning:

- LayerNorm normalizes each sample using only that sample's own features.
- BatchNorm normalizes using statistics computed *across the batch*. With a batch of 2048 drawn from a 10-million-transition buffer, those statistics describe the actual data distribution the critic must fit. That is closer to whitening the inputs, which flattens the loss surface and lowers its **condition number**.

In other words, BatchNorm only pays off *because* the batch is huge and the buffer is diverse. It is a choice that follows from pillar 1. In the standard SAC regime, with a batch of 256 from a much narrower buffer, it would be a worse choice.

Cross-batch value prediction

This is the trick that makes BatchNorm safe in a Bellman backup, and it is the single most satisfying "aha" in the paper. It comes from CrossQ.

The problem: BatchNorm normalizes using whatever batch is currently passing through. If you compute Q(s, a) in one forward pass and $\bar{Q}(s', a')$ in a second, separate forward pass, the two passes use *different* batch statistics. You then subtract one from the other in the Bellman equation. You are comparing a prediction and a target that were measured on two different rulers. The Bellman equation quietly stops being self-consistent, and training degrades or diverges.

The fix: concatenate the current-state transitions and the next-state transitions into a single batch, push them through in one forward pass, then split the outputs. Both now share one set of normalization statistics, and the comparison is valid again.

> **A USEFUL THING TO KNOW FOR QUESTIONS**
>
> CrossQ, the source of this trick, went further and removed target networks entirely, arguing that with proper normalization they are unnecessary. FlashSAC does *not* do that. It keeps exponential-moving-average target critics with $\tau = 0.01$. So it is a partial adoption, and someone will ask why. The paper does not explain the choice, which is a fair thing to say plainly.

Distributional critic with a fixed support

Instead of predicting a single number, each critic predicts a **probability distribution** over 101 possible return values, called **atoms**, spaced evenly over the fixed interval $[-5, 5]$. The network

outputs 101 probabilities. The Bellman target is computed, projected back onto that same grid of atoms, and the critic is trained with cross-entropy loss, the same loss used for image classification. This is the C51 formulation.

Why this helps stability:

- **Bounded gradients.** Cross-entropy on a softmax has gradients that are bounded by construction. Squared error does not: a target that is wrong by 1000 produces a gradient proportional to 1000. One bad bootstrapped target can therefore wreck a squared-error critic in a single step, and cannot do the same to a classification critic.
- **A smoother optimization landscape.** Predicting a distribution over a fixed grid is a better-conditioned learning problem than regressing an unbounded real number.
- **Robustness to noisy targets.** A distribution can express uncertainty about the return instead of being forced to commit to a point estimate that is partly noise.

Adaptive reward scaling, and why it is the quiet hero

A fixed support of $[-5, 5]$ only works if returns actually land inside it. The benchmark spans reward scales from about 1 (DeepMind Control) to about 18,000 (MuJoCo HalfCheetah). Something must reconcile that. Rather than centring returns or rescaling the loss, FlashSAC normalizes the *reward itself*. It tracks the running variance of the discounted return, σ^2, and the largest return magnitude seen so far, $G_{max,seen}$, and divides:

$$\bar{r}_t = r_t \,/\, \max\left(\sqrt{(\sigma^2 + \varepsilon)} \,,\;\; G_{max,seen} \,/\, G_{max,support} \right)$$

Read the denominator as two safeguards and take whichever is stricter. The first term puts returns on a roughly unit scale. The second is a safety valve: if observed returns start to exceed what the support can represent, the ratio grows above 1 and shrinks the rewards further, pulling returns back inside the grid.

> **WHY THIS DESERVES MORE ATTENTION THAN IT GETS**
>
> This equation is the mechanism that lets a *single* hyperparameter configuration work across more than 60 tasks with wildly different reward magnitudes. The paper's headline claim of "one config for everything" rests on it. It is easy to skim past as a normalization detail, and it is arguably the most practically valuable single line in the method.

Weight normalization

After every gradient step, each weight vector in the network is projected back onto the unit sphere, and each normalization parameter vector is projected to norm $\sqrt{d}$. In the paper's phrasing, the network is forced to encode information through *direction* rather than *scale*.

Two separate benefits, and the second is much less obvious than the first.

1. **It caps Q-value sensitivity.** Large weights mean a small input change can swing the output a lot, which magnifies estimation errors as they propagate through the bootstrap.
2. **It fixes the effective learning rate.** In a network with normalization layers, the function computed is invariant to the scale of the weights: doubling a weight vector before a normalization layer changes nothing about the output. But it does change the *gradient geometry*. The effective step size in function space scales roughly as $1 / \|w\|^2$. Since weight norms grow monotonically during training, an unconstrained network is silently annealing

its own learning rate towards zero. It looks like convergence, but it is paralysis. Projecting onto the sphere holds the effective learning rate constant for the whole run.

WATCH OUT: NOT EVERY COMPONENT PULLS EQUAL WEIGHT

The paper itself reports that weight normalization alone produces only modest gains, and that it is retained mainly for robustness in sample-limited regimes. Be honest about this in a presentation. The big movers in the architectural ablation are the residual blocks, BatchNorm and the distributional critic.

4.3 Pillar 3: exploration

Unified entropy target

SAC v2 asks you to specify a target entropy $\bar{H}$ in nats, and offers the heuristic $\bar{H} = -|A|$. That number has no physical meaning to a roboticist, and it does not transfer across embodiments with different action dimensions. FlashSAC instead asks you to specify a target *action standard deviation* σ_{tgt}, and derives the entropy from it using the closed form for a diagonal Gaussian:

$$\bar{H} = \tfrac{1}{2}\,|A|\,\log\left(2\pi e\,\sigma^2_{tgt}\right)$$

The formula is just the differential entropy of a Gaussian, summed over independent dimensions. The practical consequence is that the target now scales linearly with action dimension automatically, so one setting covers a 6-joint arm and a 29-joint humanoid. The paper uses $\sigma_{tgt} = 0.15$ everywhere.

DERIVED: WHAT 0.15 MEANS, COMPARED TO SAC'S HEURISTIC

With $\sigma_{tgt} = 0.15$: $2\pi e \approx 17.08$, times 0.0225 gives 0.3843, whose natural log is -0.956, halved gives -0.478. So FlashSAC's target is $\mathbf{\bar{H} \approx -0.478 \cdot |A|}$.

Now invert SAC v2's heuristic $\bar{H} = -|A|$. Per dimension that is -1 nat, so $\tfrac{1}{2}\log(2\pi e\sigma^2) = -1$, giving $\sigma^2 = e^{-3}/2\pi \approx 0.00792$, so $\boldsymbol{\sigma \approx 0.089}$.

So FlashSAC deliberately holds roughly 70 percent more exploration noise per joint than the classic SAC heuristic, and it expresses that choice in units a robotics engineer can picture: actions live in $[-1, 1]$ after the tanh, so 0.15 is 7.5 percent of the full range of travel. That is a far more meaningful dial than "minus one nat per dimension".

Noise repetition

Independent random noise added at every timestep is close to useless on a physical system, because the robot's own dynamics act as a low-pass filter. Noise that changes direction 50 times a second averages out to nothing. The robot jitters in place instead of going somewhere it has never been.

The standard fixes are temporally correlated noise processes: Ornstein-Uhlenbeck noise, or pink noise. Both require maintaining a correlated noise process *per environment*. With 1024 environments that is a real memory and compute cost, and it is why those methods are rarely used in massively parallel settings.

FlashSAC's alternative, **noise repetition**, is deliberately crude and almost free. Sample a noise vector ε from a standard normal, use it for action selection, and *hold it fixed* for k consecutive

steps. Then resample. The repeat length k is drawn from a Zeta distribution, $P(k) \propto k^{-s}$, with s = 2 and a cap of 16. The per-environment state is one vector and one integer counter. This is the continuous-control version of the temporally-extended epsilon-greedy idea from Dabney et al.

DERIVED: HOW LONG DOES THE NOISE ACTUALLY PERSIST?

For a Zeta distribution with s = 2 truncated at 16, the mean repeat length is $(\Sigma_{k=1..16}\ 1/k) / (\Sigma_{k=1..16}\ 1/k^2) \approx 3.381 / 1.584 \approx$ **2.1 steps**. So most of the time the noise barely persists at all, but the power-law tail means that occasionally it holds for the full 16 steps, producing a genuinely committed exploratory manoeuvre. "Mostly twitch, occasionally commit" is the right mental picture, and the heavy tail is doing the useful work.

4.4 The complete hyperparameter tables

These are reproduced in full because "all the exact details" means these numbers. Table A is the GPU-simulator configuration. Tables B and C list only what changes.

Table A: GPU-based simulators (IsaacLab, MuJoCo Playground, ManiSkill, Genesis)

Group	Hyperparameter	Value
Common	Parallel environments	1024
	Replay buffer capacity	10,000,000
	Batch size	2048
	Update-to-data ratio	2 per 1024 new transitions (Table 9 of the paper prints 2/2048)
	TD steps (n-step)	1
Actor	Number of residual blocks	2
	Hidden dimension	128
	Update delay	2 (actor updated every second critic update)
Critic	Number of residual blocks	2
	Hidden dimension	256
	Target critic momentum τ	0.01
	Number of critics	2 (minimum taken)
	Value prediction type	Categorical (distributional)
	Support and number of bins	$[-5, 5]$, 101 atoms
Temperature	Entropy target σ_{tgt}	0.15
	Initial α	0.01
Optimizer	Optimizer and momenta	Adam, $(\beta_1, \beta_2) = (0.9, 0.999)$
	Learning rate schedule	Cosine decay
	Learning rate, initial	3×10^{-4}

Group	Hyperparameter	Value
	Learning rate, final	1.5×10^{-4}
Noise repeat	Zeta distribution exponent s	2
	Maximum repeat length	16
Discount	γ	Matched to simulator default: 0.99 for IsaacLab, 0.97 for Playground

The discount factor is the *only* hyperparameter the paper varies across tasks in the GPU benchmark. Everything else is held constant across all 25 tasks.

Table B: CPU-based simulators (MuJoCo, DMC, HumanoidBench, MyoSuite)

Hyperparameter	Value	Change from Table A
Parallel environments	1	from 1024
Replay buffer capacity	1,000,000	from 10M
Batch size	512	from 2048
Update-to-data ratio	1	from 2/1024
TD steps (n-step)	1	unchanged

Table C: vision-based RL (DMControl from pixels)

Hyperparameter	Value	Change from Table A
Parallel environments	1	from 1024
Replay buffer capacity	1,000,000	from 10M
Batch size	256	from 2048
Update-to-data ratio	0.5	from 2/1024
TD steps (n-step)	3	from 1
Action repeat	2	new
Frame stack	3 frames, 84 × 84 × 9	new
Encoder layers	4 (3 convolutional plus a linear bottleneck)	new
Encoder channels	32	new
Encoder output feature dimension	50	new

THE MOST IMPORTANT THING THESE THREE TABLES TELL YOU

The *architecture* is identical across all three regimes. Only the data knobs move. And notice that in the single-environment CPU setting the UTD goes back to 1, exactly like standard SAC. So "use a very low UTD" is **not** the lesson of the paper. The lesson is "match the UTD to your data throughput, and make the architecture strong enough that you are allowed to." If a reading group takes away only "low UTD is better", they have misread it.

4.5 What FlashSAC deliberately did not change

Listing what stayed the same is as clarifying as listing what changed, because it establishes that this really is SAC and not a new algorithm wearing SAC's name.

- The maximum-entropy objective, unchanged.
- The soft Bellman backup, including the $-\alpha \log \pi$ entropy term in the target.
- Clipped double Q-learning with two critics and the minimum taken.
- Exponential-moving-average target critics.
- The reparameterized actor gradient flowing through the critic.
- Automatic temperature tuning via the thermostat loss on α.
- A tanh-squashed Gaussian policy.
- Adam as the optimizer, at the familiar 3×10^{-4} starting rate.

If you wrote out the update equations for SAC v2 and FlashSAC side by side, they would be the same equations. Everything that differs is in *what the function approximator looks like*, *what its output represents*, *where the data comes from*, and *how often you step*.

Part 5 FlashSAC versus SAC v2: the definitive comparison

5.1 The master difference table

This is the reference table. Every row is a real, checkable difference.

Component	SAC v2	FlashSAC	Why the change
A. Identical (the learning rule itself)			
Objective	Maximum entropy	Same	Entropy keeps the buffer diverse, which matters more at scale, not less.
Critic target	Soft Bellman with $-\alpha \log \pi$	Same	—
Overestimation control	2 critics, minimum taken	Same	—
Target networks	EMA copies of both critics	Same mechanism	Kept even though CrossQ argued normalization makes them removable.
Actor gradient	Reparameterized, through the critic	Same	—
Temperature	Learned automatically	Same mechanism	—
Policy family	tanh-squashed Gaussian	Same	—
B. Data and compute regime			
Parallel environments	1	1024 (4096 for sim-to-real)	Coverage of the state-action space is the whole reason to be off-policy.
Replay buffer	1M	10M	Preserves rare long-tail transitions that a small buffer overwrites first.
Batch size	256	2048	Saturates the GPU; also supplies the batch statistics BatchNorm needs.
Update-to-data ratio	1	2 per 1024 transitions	512 times fewer updates. Cuts wall-clock and cuts bootstrap error recycling.
Implementation	Not specified	JIT compiled, mixed precision	Worth 5 to 10 percent of wall-clock on its own.
C. Network architecture			
Size	2 hidden layers of 256, about 0.1 to 0.2M parameters	6 layers, 2.5M parameters total	Capacity to fit a value function over 10M diverse transitions.
Actor vs critic size	Same for both	Actor width 128, critic width 256	The actor runs on the robot at 50 Hz, so keep it cheap; the critic is trained only.

Component	SAC v2	FlashSAC	Why the change
Block structure	Plain feedforward	Inverted residual blocks, d → 4d → d, with skip connection	Residual paths keep the gradient norm bounded, which is what allows depth.
Normalization	None	Pre-activation BatchNorm at every nonlinearity	Replay data is non-stationary; without it, units saturate and stop learning.
Batch statistics in the backup	Not applicable	Cross-batch: current and next states concatenated into one forward pass	Without this, prediction and target are normalized differently and the Bellman equation breaks.
Final feature norm	Unbounded	RMSNorm before the value head	Stops out-of-distribution inputs producing unbounded Q values.
Weight magnitude	Unconstrained, grows during training	Projected onto the unit sphere after every step	Caps Q sensitivity and stops the effective learning rate decaying to zero.
D. What the critic predicts			
Output	One scalar per critic	101-way categorical distribution over [−5, 5]	Bounded gradients and a better-conditioned loss than unbounded regression.
Loss	Mean squared error	Cross-entropy against the projected Bellman target	One outlier target cannot produce an enormous gradient.
Reward handling	Raw rewards	Divided by running return statistics, with a support-overflow safety valve	This is what makes one hyperparameter set work across 60+ tasks.
E. Exploration			
Entropy target	$\bar{H} = -\lvert A\rvert$ nats	Specified as an action std $\sigma_{tgt} = 0.15$, giving $\bar{H} \approx -0.478\lvert A\rvert$	Physically meaningful and transfers across embodiments with no retuning.
Action noise over time	Fresh independent sample every step	Noise vector held for k steps, k drawn from Zeta(s = 2), capped at 16	Independent per-step noise is filtered out by robot dynamics and explores nothing.
F. Smaller differences worth knowing			
Actor update frequency	Every critic update	Every second critic update	TD3-style delay: let the critic settle before the actor chases it.
Target smoothing τ	0.005	0.01	Targets track twice as fast, affordable because there are far fewer updates.
Learning rate	Constant 3×10^{-4}	Cosine decay, 3×10^{-4} to 1.5×10^{-4}	Standard large-batch training practice.
Initial α	1.0	0.01	Starts near-greedy and lets the thermostat raise it if needed.
n-step returns	1	1 for state, 3 for vision	Longer credit assignment where rewards are sparser and frames are stacked.

5.2 The five differences that actually matter

If you only have time to say five things in a presentation, say these, in this order.

1. **The update-to-data ratio is 512 times lower.** This is the most radical departure and the least intuitive. Everything else exists to make it survivable.
2. **The critic is a classifier over normalized returns, not a regressor over raw returns.** This single change buys bounded gradients, robustness to bad targets, and task-agnostic hyperparameters all at once.
3. **Every norm in the network is bounded by construction.** Weights on a sphere, features through RMSNorm, gradients through residual paths, activations through BatchNorm. This is what lets a 2.5M-parameter critic train under bootstrapping at all.
4. **BatchNorm is used, and it is made safe by one concatenation.** The cross-batch trick is one line of code and it is the difference between BatchNorm helping and BatchNorm destroying the Bellman equation.
5. **Exploration is expressed in physical units and is temporally correlated.** An action standard deviation instead of a nat count, and noise that persists for a couple of steps instead of resampling every step.

5.3 Side-by-side pseudocode

SAC v2, one iteration	FlashSAC, one iteration
1. Step 1 environment, store (s,a,r,s') in a 1M buffer. 2. Sample a batch of 256. 3. a' ~ pi(.|s'). 4. Forward pass 1: Q_target(s', a'). 5. Forward pass 2: Q(s, a). 6. y = r + gamma * (min_j Qbar_j(s',a') - alpha*logpi(a'|s')). 7. Critic loss = MSE(Q(s,a), y). 8. Actor loss = alpha*logpi(a|s) - min_i Q_i(s,a). 9. Temperature loss on alpha. 10. Adam step on all three. 11. EMA target update, tau = 0.005. *Repeat once per environment step.*	1. Step 1024 environments; if the noise counter has expired, resample epsilon, else reuse it. Store 1024 transitions in a 10M buffer. 2. Twice per 1024 transitions, sample a batch of 2048. 3. a' ~ pi(.|s'). 4. **Concatenate** (s,a) and (s',a') into one batch. 5. **One** forward pass through the normalized residual network; split the outputs. Both share BatchNorm statistics. 6. Normalize rewards by running return statistics. 7. Build the categorical Bellman target and project it onto the 101 atoms. 8. Critic loss = cross-entropy. 9. Actor loss as in SAC, but only on every second update. 10. Temperature loss on alpha, against the sigma-derived target entropy. 11. Adam step, cosine-decayed learning rate. 12. **Project every weight vector back onto the unit sphere.** 13. EMA target update, tau = 0.01. *Repeat twice per 1024 environment steps.*

5.4 A worked numeric example

Take one FlashSAC GPU benchmark run: 50 million environment steps. How much work is that, and what would SAC v2 have done?

Quantity	FlashSAC	SAC v2 at UTD 1
Collection rounds	50M / 1024 ≈ 48,828	50M single steps
Gradient updates performed	48,828 × 2 ≈ **97,700**	**50,000,000**
Ratio of updates	FlashSAC does **512 times fewer** gradient updates for the same amount of experience.	
Samples drawn from the buffer	97,700 × 2048 ≈ 200 million	50M × 256 = 12.8 billion
Ratio of samples processed	FlashSAC processes **64 times fewer** samples through the network.	
Parameters per forward pass	about 2.5M	about 0.2M
Approximate total network compute	64 times fewer samples through a roughly 12 times larger network means roughly **5 times less total arithmetic**, while holding far more capacity.	

That last row is the whole scaling argument in one number. FlashSAC is not buying performance with compute. It is spending noticeably *less* arithmetic than a naive SAC would, and putting what it does spend into a bigger model taking better-informed steps rather than a small model taking millions of noisy ones.

Part 6 The experiments

6.1 Protocol, hardware, and how wall-clock was measured

Scope: more than 60 tasks across 10 simulators, spanning state-based control on GPU simulators, state-based control on CPU simulators, vision-based control, and sim-to-real humanoid locomotion. Benchmark hardware is a single RTX 5090 GPU with an AMD Ryzen 9 9950X3D CPU.

Setting	Tasks	Baselines	Training budget
GPU, state-based	25 (IsaacLab, MuJoCo Playground, ManiSkill3, Genesis)	PPO from RSL-RL, FastTD3	50M steps for off-policy, 200M for PPO
CPU, state-based	40 (MuJoCo, DMC, MyoSuite, HumanoidBench)	PPO, XQC, SimbaV2, TD-MPC2, MR.Q	1M steps, 4M for PPO
Vision	8 (DMControl from pixels)	DrQ-v2, MR.Q (also XQC and DrM in the appendix)	1M steps, action repeat 2
Sim-to-real	Unitree G1, 29 DoF, blind locomotion	PPO with the DreamWaQ sim-to-real pipeline	About 4 hours on an A100, 4096 environments

The GPU tasks are split by dimensionality, and this split is the paper's central experimental variable:

- **Low-dimensional, 15 tasks**: Franka gripper manipulation, AnyMal-C and AnyMal-D and Unitree Go2 quadruped locomotion.
- **High-dimensional, 10 tasks**: Allegro and Shadow Hand cube reorientation (16 and 20 action dimensions), and Unitree G1, H1 and Booster T1 humanoid locomotion (19 to 37 action dimensions, up to 310-dimensional observations).

WATCH OUT: READ APPENDIX A BEFORE YOU PRESENT THE WALL-CLOCK PLOTS

The compute-time axes are not raw stopwatch numbers. The paper's stated protocol splits total runtime into two parts:

- *Environment interaction time* is measured separately for each environment. Fair, since it depends only on the simulator.
- *Algorithm update time* is profiled on **one representative environment** (MuJoCo Humanoid-v4 for state-based, DMC Walker-run for vision) and that single measurement is then **reused for every other environment**.

Total wall-clock is then the sum of the two. This is a reasonable way to remove hardware noise from the comparison, but it means the plots are reconstructed rather than measured end to end, and it assumes update cost does not vary with observation dimension. It plainly does vary: DMC dog-run has a 223-dimensional observation, cartpole has 5. The paper acknowledges this and judges the effect negligible. This is the sharpest methodological question in the paper and you should raise it yourself rather than be caught by it.

6.2 GPU state-based results

- **Low-dimensional tasks: parity.** The paper's own wording is that FlashSAC "slightly outperforms" PPO. Say this honestly. It confirms rather than overturns the prior consensus that on-policy methods are fine when the action space is small and samples are cheap.
- **High-dimensional tasks: a clear and consistent win.** Across dexterous manipulation and humanoid locomotion, FlashSAC reaches higher final return in substantially less wall-clock time, despite PPO being given four times as many environment steps and roughly three times the compute.
- **Against FastTD3, the story is stability, not speed.** FastTD3 is also a wall-clock-optimized off-policy method, but the paper reports it failing outright or badly underperforming on several tasks (Go2 Walk, Franka Pull Cube). FlashSAC converges on all of them. Where both converge, FlashSAC reaches a higher asymptote, with the largest gaps on humanoid locomotion where the extra model capacity pays off most.

6.3 CPU state-based results

This is the sample-efficiency regime: one environment instance, so data throughput is low and every sample counts. FlashSAC is compared not against PPO alone but against the strongest current sample-efficient methods, including XQC (batch-normalized off-policy), SimbaV2 (normalized scaling), TD-MPC2 (model-based with planning) and MR.Q (model-free with a model-based representation objective).

The reported outcome is that FlashSAC matches or beats all of them across representative tasks, using a single configuration and no task-specific tuning, while the baselines are tuned per task. PPO performs particularly poorly here, which is expected: without data reuse it simply cannot compete on a one-million-step budget.

The significance of this section is broader than the numbers. It shows the design generalizes beyond the massively parallel regime it was designed for. That is a real robustness result, and it is the answer to anyone who suspects the method only works because it is drowning in data.

6.4 Vision-based results

On 8 pixel-based DMControl tasks, FlashSAC matches or exceeds the baselines in final performance while converging faster in wall-clock time. The specific observations:

- DrQ-v2 is sample-efficient but unstable, and fails to converge on some tasks such as Finger Turn Hard.
- MR.Q reaches high final performance but pays for it with the extra compute of its auxiliary dynamics model.
- FlashSAC gets competitive or better results with a single hyperparameter set and no auxiliary objective.

The paper is careful to add that its stabilization techniques are *orthogonal* to representation learning, and that MR.Q's objective could be layered on top for further gains. Read that as an honest admission that vision is the weakest of the four result sections: competitive, not transformative.

6.5 Sim-to-real on the Unitree G1, in full detail

This is the headline experiment and the one that would change practice, so it is worth knowing the setup properly rather than just the numbers.

The result

Task	FlashSAC	PPO	Speedup
Stable flat-ground walking, deployed on hardware	about 20 minutes of training	about 3 hours	roughly 9×
Real stair climbing, 15 cm rise	about 4 hours	nearly 20 hours	roughly 5×

The learned flat-ground policy supports omnidirectional locomotion, forward, backward and lateral, without retraining. The real stairs used at test time (15 cm high, 60 cm wide, 1.5 m platform) had dimensions the policy never saw in training, so this is genuine generalization and not memorization of the training terrain.

The training setup

- **Robot and rates.** Unitree G1, 29 degrees of freedom, blind (no cameras or lidar). The policy outputs target joint positions at 50 Hz. A PD controller converts those into torques at 200 Hz, with per-joint gains taken from published heuristics.
- **Terrain curriculum.** Five terrain types: pyramid stairs up to 23 cm rise, random grids up to 15 cm, random uniform roughness from −2 to +4 cm, waves up to 20 cm amplitude, and pits up to 30 cm deep. Ten difficulty levels, advanced automatically once the policy successfully traverses 50 percent of the environment. The authors note that getting past level 5 is difficult without a perception module and starts to produce aggressive motions.
- **Domain randomization.** Applied at large scale alongside the curriculum, plus uniform sensor noise injected into every observation channel (for example ±0.2 rad/s on base angular velocity, ±1.5 rad/s on joint velocities).
- **Implicit system identification.** Both PPO and FlashSAC use a context estimator network (CENet, from DreamWaQ). It reads a history of proprioceptive observations and predicts the robot's base linear velocity plus a latent vector describing the current dynamics. An auxiliary loss trains the velocity prediction. This is what lets a blind robot infer, from how its own joints have been behaving, that it is on stairs rather than flat ground.
- **Asymmetric actor-critic.** The critic receives privileged information available only in simulation: ground-truth base linear velocity, foot contact states, and a 17×11 terrain height map. The actor sees only what the real robot can measure. This is standard practice and it is worth stating clearly, because it means the critic is solving an easier problem than the actor by design.
- **Symmetry augmentation.** Training batches are augmented using the robot's left-right symmetry, which the authors report improves sample efficiency and produces more natural gaits.

WATCH OUT: THE SIM-TO-REAL COMPARISON IS NOT FULLY LIKE-FOR-LIKE

Appendix Table 14 shows that PPO and FlashSAC use the **same reward structure but different reward weights**. The most consequential difference is termination handling: PPO gets a −200 penalty on early termination, while FlashSAC gets a small alive bonus and no termination penalty. The paper's justification is that the two algorithms have different learning dynamics and each needs its own shaping to produce a policy safe enough to run on real hardware, which is a real and defensible engineering concern. But it does mean the reward function is not held constant across the two arms of the comparison. Add to this: one robot, one platform, qualitative success reporting rather than success-rate tables over repeated trials, and mixed hardware (an A100 with 4096 environments here, versus an RTX 5090 with 1024 environments in the benchmarks). The direction of the result is convincing. The precise multiplier is softer than it looks.

Part 7 The analysis and ablations

All ablations are run on four IsaacLab environments: Allegro Hand cube reorientation, Shadow Hand cube reorientation, and G1 humanoid locomotion on flat and rough terrain. That is a narrow base, and worth flagging.

7.1 Off-policy versus on-policy coverage

This experiment is the mechanistic justification for the whole paper, so it deserves airtime.

They train FlashSAC for 1M steps on the Shadow Hand task with a 1M buffer, then roll out the final policy for another 1M steps to collect a purely on-policy dataset. They then plot the joint density of object y-position against fingertip joint action, for each of five fingers, for both datasets.

The result: the off-policy replay data covers a visibly broader region of the state-action space, because it accumulated experience across thousands of different behaviour policies. The on-policy data is tightly concentrated around the final policy's own distribution.

Why this matters. A critic can only be trusted where it has data. If your data is a thin thread, your value estimates off that thread are guesses. The paper's argument is that this, and not sample efficiency, is the real reason on-policy methods lose ground in high dimensions, and that achieving comparable coverage with on-policy rollouts would require enormously more data collection.

> **THE OBVIOUS OBJECTION, AND THE ANSWER**
>
> Offline RL has spent years showing that broad data coverage *far from the current policy* causes extrapolation error and value divergence. So why is coverage good here? The answer is that coverage without a bounded critic *is* the classic failure mode. Every mechanism in pillar 2 exists so the critic can be evaluated at unfamiliar state-action pairs without producing garbage. Coverage and stability are not separate benefits, they are a package: the coverage is what you want, and the bounded norms are the price of admission. Expect this question and answer it with RMSNorm, which is the most direct structural defence.

7.2 Scaling ablations (Figure 8)

Five univariate sweeps, all plotted against wall-clock time rather than environment steps, which is the right axis for the paper's claim.

Knob swept	Values tested	Finding
Replay buffer size	0.1M, 1M, 10M, 50M	Improves up to 10M by stabilizing training. 50M is *worse* in wall-clock, because recent high-quality samples get drawn less often, though it can reach a slightly higher asymptote given enough time. A genuine trade between coverage and recency.
Batch size	0.5K, 1K, 2K, 4K, 8K	Larger accelerates convergence, consistent with supervised scaling behaviour. 2K chosen as the practical default.
Network width	64, 128, 256, 512, 1024	Larger accelerates convergence. 256 chosen.

Knob swept	Values tested	Finding
Network depth	1, 2, 3, 4 blocks	Deeper accelerates convergence. 2 chosen.
Update-to-data ratio	0.5, 1, 2, 4, 8 per 1024	Reducing the UTD accelerates convergence in wall-clock. 2 chosen.

The paper's summary of these: most existing off-policy methods use small architectures purely to stay stable, which caps how fast they can converge. Once the stability mechanisms let you go bigger, the scaling behaviour that everyone knows from supervised learning becomes available.

> **WATCH OUT: THIS IS NOT A SCALING LAW**
>
> The paper repeatedly frames pillar 1 as following "scaling laws", citing Kaplan et al. But Kaplan et al. is about fitting power-law relationships between loss, model size and compute for language model pretraining. FlashSAC fits no curves and estimates no compute-optimal frontier. What it has is five univariate sweeps whose direction is consistent with the trend. That is an analogy used as a design heuristic, and it is a good one, but calling it a scaling law is generous. Say so.

7.3 The architectural ablation (Figure 9)

This is the most convincing figure in the paper and the one to spend time on. Starting from a plain MLP critic, they add one component at a time:

> MLP → + Residual → + BatchNorm → + RMSNorm → + Distributional critic → + Weight norm (= FlashSAC)

Crucially, they do not just report the score. They also track four diagnostics through training: weight norm, feature norm, gradient norm, and the **condition number** of the critic's loss landscape.

The condition number is worth explaining properly, because most people nod at it without knowing what it means. It is the ratio of the largest to the smallest curvature of the loss surface. A high condition number means the loss valley is long and extremely narrow, so a single learning rate is simultaneously too large in one direction (you bounce off the walls) and too small in another (you crawl along the floor). Gradient descent zigzags. Low condition number means a well-shaped bowl where one step size works in every direction.

The finding. As each component is added, the three norms stay bounded throughout training with no uncontrolled growth, and the condition number decreases monotonically, reaching its lowest value with the full architecture. Normalized task score rises in lockstep.

This is what turns "a bag of tricks" into an argument. The paper is not just showing that the combination scores higher, it is showing that each addition moves the specific optimization diagnostic it was supposed to move, in the predicted direction, every time. If a reading group is going to be convinced by one figure, this is it.

7.4 The exploration ablation (Figure 10)

- **Entropy target.** Swept over σ_{tgt} in $\{0.05, 0.1, 0.15, 0.2, 0.25\}$. Performance is largely insensitive; all settings converge to similar scores, with the optimum somewhere in 0.15 to 0.2. This is the paper's cheapest robustness claim, and it is a real practical selling point: one fewer thing to tune per robot.

- **Noise repetition.** Disabling it produces slower convergence and lower final aggregate scores. The paper's explanation is the one from Part 4.3: repeated noise produces coherent exploratory trajectories, while uncorrelated per-step perturbations are averaged away by the dynamics.

Part 8 Numbers worth memorizing

Quantity	Value
Scope of evaluation	60+ tasks, 10 simulators, single RTX 5090
Parallel environments / buffer / batch	1024 / 10M / 2048
Update-to-data ratio	2 gradient steps per 1024 new transitions
Derived replay ratio	each transition used about 4 times, versus about 256 for standard SAC
Network	2.5M parameters, 6 layers; actor width 128, critic width 256, 2 blocks each
Distributional critic	101 atoms on $[-5, 5]$, cross-entropy loss
Target smoothing / actor delay	$\tau = 0.01$ / actor updated every 2nd critic update
Learning rate	3×10^{-4} cosine-decayed to 1.5×10^{-4}, Adam
Entropy target	$\sigma_{tgt} = 0.15$, giving $\bar{H} \approx -0.478\|A\|$
Noise repetition	Zeta exponent 2, cap 16, mean repeat about 2.1 steps
Training budgets	50M steps off-policy vs 200M for PPO (GPU); 1M vs 4M (CPU); 1M (vision)
CPU config differences	1 environment, 1M buffer, batch 512, UTD 1
Vision config differences	batch 256, UTD 0.5, 3-step returns, frame stack 3, action repeat 2
Sim-to-real headline	20 min vs 3 h on flat; 4 h vs 20 h on stairs; 29-DoF Unitree G1
Sim-to-real setup	4096 environments, A100, policy at 50 Hz, PD control at 200 Hz, 10 terrain levels

Part 9 Limitations and the questions you will be asked

Question	How to answer it
Isn't this just a bag of engineering tricks?	Largely yes, and the paper does not pretend otherwise. The defensible contribution is the *ordering* claim: that stability is a prerequisite for the scaling regime rather than a nice-to-have. Nobody had previously shown that this combination flips the PPO-versus-off-policy verdict on real hardware. Figure 9 is the evidence that the mechanism is real and not a lucky hyperparameter.
Are the wall-clock numbers actually measured?	Only partly. See the Appendix A caveat in Part 6.1. Update time is profiled on one representative environment and reused everywhere. Raise this yourself.
Is the PPO baseline fair?	It cuts both ways, and saying so is the honest answer. In PPO's favour: it gets four times the environment steps and roughly three times the compute, and it is a strong, per-task-tuned RSL-RL implementation. In FlashSAC's favour: it runs a single configuration across every task, varying only γ, which is a much stronger generality claim than the baselines are held to.

Question	How to answer it
Why is FastSAC not benchmarked?	Good catch to make yourself before someone else does. FastSAC (the "learn sim-to-real humanoid locomotion in 15 minutes" paper) is arguably the closest competitor and is cited in the related work, but only FastTD3 appears in the result plots. The paper does not explain the omission.
Doesn't broad data coverage cause extrapolation error, per the offline RL literature?	Yes, and that is exactly why pillar 2 exists. See the box in Part 7.1. Answer with RMSNorm before the value head as the most direct structural defence.
Why does BatchNorm work here when it usually breaks RL?	Because of the cross-batch concatenation, which makes the prediction and the target share normalization statistics. And because the batch is 2048 drawn from a 10M diverse buffer, which is what makes batch statistics informative in the first place. In the standard SAC regime it would be a worse choice than LayerNorm.
Why keep target networks if CrossQ showed you can drop them?	The paper keeps EMA targets at $\tau = 0.01$ and does not explain why. State that plainly. A reasonable guess is that with a much larger critic and a distributional head they preferred not to remove a second stabilizer at the same time, but the paper does not say.
Is the sim-to-real comparison like-for-like?	Not entirely. Different reward weights per algorithm, most notably PPO's -200 termination penalty against FlashSAC's alive bonus. One robot, qualitative reporting, mixed hardware. See the box in Part 6.5.
Do scaling laws really apply?	It is an analogy used as a design heuristic, not a fitted law. No curves, no compute-optimal frontier. The five univariate sweeps are directionally consistent, which is weaker.
So is the lesson "use a very low UTD"?	No, and this is the most common misreading. In the single-environment CPU configuration the UTD goes straight back to 1. The invariant across all three regimes is the *architecture*. The UTD is a function of data throughput.
What breaks if I try this on my own problem?	Three specific dependencies. The reward normalization assumes running return statistics are meaningful, so pathological or extremely sparse reward structures need care. BatchNorm needs large batches to be informative, so it degrades if you cannot afford them. And the very low UTD only makes sense when thousands of parallel environments are feeding it.
Is the debate settled?	No, and it is worth ending on this. The on-policy side is not standing still. The paper's own related-work section cites recent work on stronger trust-region guarantees, growing action spaces, and pathwise gradient estimators for policy optimization. None of those newer on-policy methods appear in the benchmark. The comparison is against PPO, which is the field's default, not necessarily the field's best on-policy method today.

TWO SMALL INTERNAL INCONSISTENCIES

First, the update-to-data ratio is printed as 2/1024 in Section 4.1 and 2/2048 in Table 9. Second, the task counts do not quite reconcile: Section 5.2 says 40 CPU tasks, while the appendix tables list 5 MuJoCo + 10 DMC + 14 HumanoidBench + 10 MyoSuite = 39. Neither affects any conclusion, but a careful reader may spot them and it is better to have an answer ready.

Part 10 Where each idea came from

FlashSAC is explicitly a synthesis, and naming the parents makes the contribution legible rather than diminishing it. This table is a good single slide.

Component in FlashSAC	Source	What FlashSAC took
Base algorithm	SAC (Haarnoja et al., 2018)	Maximum-entropy objective, soft Bellman backup, learned temperature
Two critics, minimum taken	TD3 (Fujimoto et al., 2018)	Clipped double Q-learning; also the delayed actor update
BatchNorm with shared statistics	CrossQ (Bhatt et al., 2024)	Pre-activation BatchNorm and the cross-batch concatenation trick
Residual blocks, weight normalization	Simba and SimbaV2 (Lee et al., 2024, 2025)	Normalized architecture for scaling RL networks. Note: the same first author is a joint corresponding author on FlashSAC
Condition-number diagnostics, batch-norm off-policy	XQC (Palenicek et al., 2026)	Well-conditioned optimization framing; two XQC authors are co-authors here
Massively parallel off-policy throughput	FastTD3 and FastSAC (Seo et al., 2025)	The high-throughput regime, and the baseline to beat
Categorical distributional critic	C51 (Bellemare et al., 2017)	Atoms, projection of the Bellman target, cross-entropy loss
Temporally extended exploration	epsilon-z-greedy (Dabney et al., 2020)	Zeta-distributed repeat lengths
Inverted bottleneck block	MobileNet (Howard, 2017), Transformer FFN (Vaswani et al., 2017)	The d to 4d to d expansion shape
Residual connections, RMSNorm	ResNet (He et al., 2016), RMSNorm (Zhang and Sennrich, 2019)	Gradient path and feature-norm bounding
Scaling framing	Kaplan et al. (2020)	Bigger model, bigger batch, fewer steps
Sim-to-real pipeline	DreamWaQ (Nahrendra et al., 2023)	Context estimator network for implicit system identification; a DreamWaQ author is a co-author here
Asymmetric actor-critic	Pinto et al. (2017)	Privileged critic observations in simulation only
Terrain curriculum	Rudin et al. (2022)	Game-inspired progressive terrain difficulty

A compact way to summarize it: *FlashSAC is FastTD3's throughput plus SimbaV2's normalized architecture plus CrossQ's BatchNorm consistency plus C51's distributional critic plus epsilon-z-greedy's noise repetition, integrated into one configuration that is never retuned.* The integration is the contribution, and the paper is honest about that.

Part 11 Glossary

Term	Plain-English meaning
Actor	The policy network. Turns a state into an action.
Critic	The value network. Estimates how much future reward an action will earn.
Asymmetric actor-critic	Giving the critic extra information during training that the real robot will never have, such as a terrain height map. Only the actor is deployed.
Atoms	The fixed grid of possible return values a distributional critic assigns probabilities to. Here 101 of them, evenly spaced on $[-5, 5]$.
BatchNorm	Rescales activations using statistics computed across the whole batch, so they stay in a healthy range.
Bootstrapping	Training the critic against a target built from the critic's own predictions. Efficient, but it recycles errors.
Clipped double Q-learning	Keeping two critics and always using the smaller estimate, to counteract the systematic overestimation that Q-learning otherwise develops.
Condition number	How badly shaped the loss surface is. High means a long narrow valley where no single learning rate works well in all directions.
Deadly triad	Function approximation plus bootstrapping plus off-policy data. Each is fine alone; together they can make training diverge.
Distributional critic	A critic that predicts a probability distribution over possible returns instead of a single number.
Domain randomization	Randomizing simulation parameters such as mass, friction and latency so the policy cannot overfit to one exact physics model.
Effective learning rate	The step size you actually take in function space, as opposed to the number in your optimizer. In a normalized network it shrinks as weights grow.
Entropy	How spread out a probability distribution is. High entropy policy means varied actions; low entropy means predictable ones.
Extrapolation error	The confident nonsense a network produces when asked about inputs unlike anything in its training data.
Inverted bottleneck	A block that expands the feature width, does its work, then contracts back. Capacity in the middle, narrow interfaces.
Off-policy	Learning from data collected by policies other than the current one, usually from a replay buffer.
On-policy	Learning only from data collected by the current policy, then discarding it.
Plasticity loss	A network gradually losing the ability to learn new things, often through saturated or permanently inactive units.
Privileged information	Simulator-only data given to the critic, such as ground-truth velocity or contact states.
Replay buffer	The memory of past transitions that an off-policy algorithm samples from.
Reparameterization trick	Writing a random action as a deterministic function of the state and a fixed noise sample, so gradients can flow through the sampling step.

Term	Plain-English meaning
RMSNorm	Dividing a feature vector by its own root-mean-square magnitude, which puts it on a fixed-radius sphere.
Residual connection	Adding a block's input to its output, giving the gradient a direct path backwards.
Sim-to-real gap	The mismatch between simulated and real physics that makes a policy trained in simulation fail on hardware.
Soft Bellman backup	The standard Bellman update with an entropy bonus added to the target.
Target network	A slowly updated copy of the critic used to compute targets, so the critic is not chasing something moving as fast as itself.
Temperature α	The exchange rate between reward and randomness in maximum-entropy RL. Learned automatically in SAC v2.
Terrain curriculum	Automatically raising terrain difficulty as the policy succeeds, so it is never asked for too much too early.
UTD (update-to-data ratio)	Gradient updates performed per new transition collected. The paper's central dial.
Weight normalization	Forcing weight vectors to unit length after every step, so the network encodes information in direction rather than magnitude.

Prepared as a study companion to arXiv:2604.04539v2. Statements of fact about the method, hyperparameters and results are taken from the paper and its appendices. Boxes marked "Derived" contain arithmetic performed on the paper's stated values. Boxes marked "Watch out" flag caveats and internal inconsistencies identified during review.